Reproducible delivery for a task tracking API

DD2482 DevOps 2026 Project report

Hasan Kalzi (kalzi@kth.se) and Love Lingren (lolindgr@kth.se)

Revised 19 October 2026

https://github.com/Hasan-Kalzi/task-tracking-project-devops

We built a task tracking API with an integrated GitHub Actions, Docker and Ansible workflow. Release v0.1.1 passed automated delivery. FastAPI and SQLite provide the application. GitHub Actions tests and publishes a Docker image archive, and Ansible applies it to an Ubuntu VM. Hasan verified VM provisioning, unchanged repeats for v0.1.0, container recreation persistence, and an upgrade to v0.1.1 followed by manual rollback to v0.1.0. The original saved task remained unchanged [1-4].

Architecture and application behaviour

FastAPI creates, lists, retrieves and completes tasks. Pydantic validates request types and limits. SQLite uses bound SQL parameters and short transactions; startup rejects an unsupported schema. Tests cover invalid input, missing tasks, filtering, pagination and storage faults. The health endpoint queries storage and returns the selected application version, allowing deployment to check both availability and release selection.

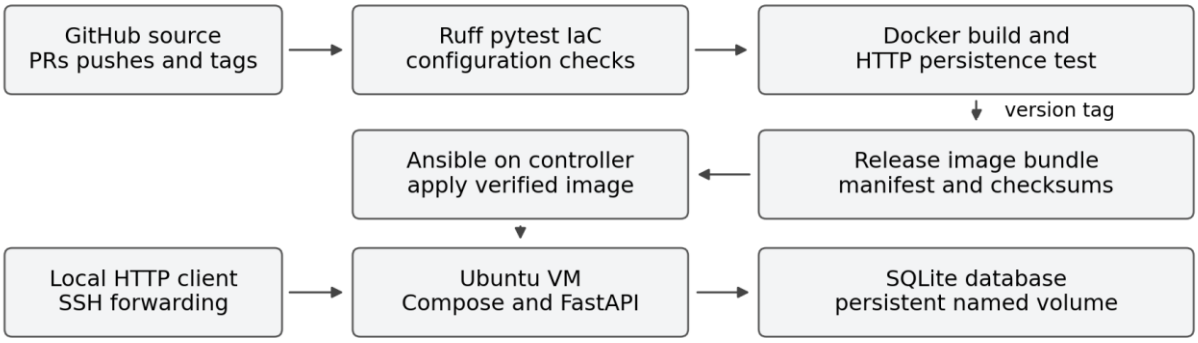


Figure 1 Source verification, versioned delivery and VM runtime.

Continuous integration and delivery

CI runs on pushes and pull requests. It installs hash-locked dependencies, checks Ruff lint and formatting, runs pytest, validates Ansible and Compose configuration, and builds the Docker image. A disposable Compose project tests HTTP operations, completes a task, recreates the container and confirms that its named volume retains the task.

Tags matching vMAJOR.MINOR.PATCH trigger delivery. The gate repeats these checks, builds one Linux/amd64 image, tests it and saves the same image without rebuilding. The release contains image.tar.gz, a deployment bundle, release.json and SHA256SUMS. The manifest records the source commit, image ID, version and architecture. Packaging and publication follow a successful gate; applying the release to the VM is a separate Ansible operation [3].

Infrastructure and reproducible operation

A dedicated WSL Python 3.12.14 environment runs Ansible 2.19.14. Its target is a separate Multipass Ubuntu 24.04.5 x86_64 VM with 2 CPUs, 2 GB RAM and a 12 GB disk. VM creation, SSH access, Python and a sudo-capable account are prerequisites. Ansible checks the platform, installs Docker and Compose, enables the daemon and creates /opt/task-api. The immediate unchanged provisioning repeat reported changed=0.

Deployment copies and loads the image archive without rebuilding. The corrected playbook compares its configuration digest with the verified manifest's image_id and rejects conflicting reuse of a release tag. It writes Compose configuration and APP_VERSION into .env, waits for container health and verifies the HTTP version. The same corrected playbook was used when selecting either v0.1.0 or v0.1.1 [4,6].

Compose runs one API container with persistent SQLite storage at /data/tasks.db. It uses UID/GID 10001, a read-only root filesystem, dropped capabilities and no-new-privileges. HTTP binds to VM loopback and is reached through SSH forwarding. The real inventory and private keys stay outside tracked files.

Design decisions and trade offs

SQLite avoids a separate database service and credentials. It suits this small single-instance API, but concurrent writes and availability are limited. PostgreSQL would improve concurrent operation while adding another service and recovery procedures. Standard-library SQL keeps the current schema simple; future schema changes require an explicit migration strategy.

Docker archives allow delivery without a registry login or cloud account, at the cost of larger downloads and explicit distribution. Ansible applies the tested artefact. Manual VM deployment avoids storing target SSH credentials in GitHub Actions; a reachable runner or pull-based agent could automate that operation.

Python packages, Python and GitHub Actions revisions are pinned; package hashes are required. The base image uses a version tag and Ubuntu packages track updates, so the environment is not guaranteed to remain byte-identical. Ruff provides quality automation; no vulnerability scan is claimed.

Corrections during integration

Windows installation exposed a missing conditional colorama dependency; universal lock resolution added its version and hashes. Love corrected version forwarding into Compose, and Ansible also required APP_VERSION explicitly. Direct WSL SSH timed out, so a verified-host-key connection used Windows TCP forwarding. Commit 7414ff7 corrected image identity verification for Docker image-store compatibility. The original v0.1.0 bundle predates that playbook fix [4,6].

Verification results

CI and delivery passed for v0.1.1 at commit 7414ff7 on 11 October [2,3]. The VM results below were executed and confirmed by Hasan; their dated record links the versions, corrected playbook and retained controller logs [4].

Check	Result	Evidence
CI and delivery	Passed	25 tests, Ruff, IaC/Compose validation, Docker build and HTTP/persistence checks [2,3].
Release assets	Published	v0.1.1 has four assets: image, deployment bundle, manifest and checksums [3].
VM provisioning	Confirmed	Ubuntu 24.04.5 x86_64; immediate repeat changed=0, unreachable=0, failed=0 [4].
Archive deployment	Confirmed	v0.1.0 health/version passed; unchanged repeat changed=0, unreachable=0, failed=0 [4].
Recreation persistence	Confirmed	The completed task was identical after forced recreation; container IDs changed [4].
Upgrade and rollback	Confirmed	v0.1.0 to v0.1.1 to v0.1.0; the original saved task remained unchanged [4].
Failure blocking	Passed	v0.1.2: 1 intentional failure and 25 passes; packaging and publication were skipped. No release existed at verification [5].

The negative control used an unmerged experiment branch on 17 October [5]. Rollback used the corrected playbook and retained the existing SQLite volume [4,6]. An upstream TestClient deprecation warning remains.

AI assistance and collaboration

The students carried out the main work by understanding the topic, defining the project goals, planning the implementation, and making the key technical decisions. AI tools, including Codex and ChatGPT, were used as supporting resources to assist with completing tasks, generating initial code and configurations, correcting errors, debugging issues, and checking the implementation. Hasan managed the public branches and pull requests, while Love worked on CI improvements and versionforwarding issues. PR #4 documents the review and reproduction process. AI-assisted commits are labelled [Codex], and docs/ai-use.md describes how AI was used, the corrections made by the students, and the verification limitations. Both students reviewed the final output, ensured it met the project requirements, and are responsible for understanding and explaining the complete system

Limitations and next improvements

The prototype uses synthetic data and one instance, with no authentication, TLS, public hosting or high availability. Manual rollback between the tested compatible releases preserves current data; it cannot reverse schema changes or recover deleted tasks. There is no automatic rollback. Checksums detect corruption but do not authenticate releases. Future work includes signed artefacts, digest-pinned base images, database migrations and tested backup restoration.

Evidence and references

[1] Public source repository
[2] Passing CI 38172535137
[3] Successful delivery Release v0.1.1
[4] Rollback and VM record Evidence commit 96c1a50

[5] Controlled failed delivery Evidence record
[6] Deployment correction commit 7414ff7
[7] DD2482 Project criteria